

E-Commerce System

A Spring Boot REST API for managing an e-commerce system including users, products, categories, merchants, merchant stock, purchases, balance operations, discounts, and administrative operations.

User

Product

Category

Merchant

MerchantStock

About Me

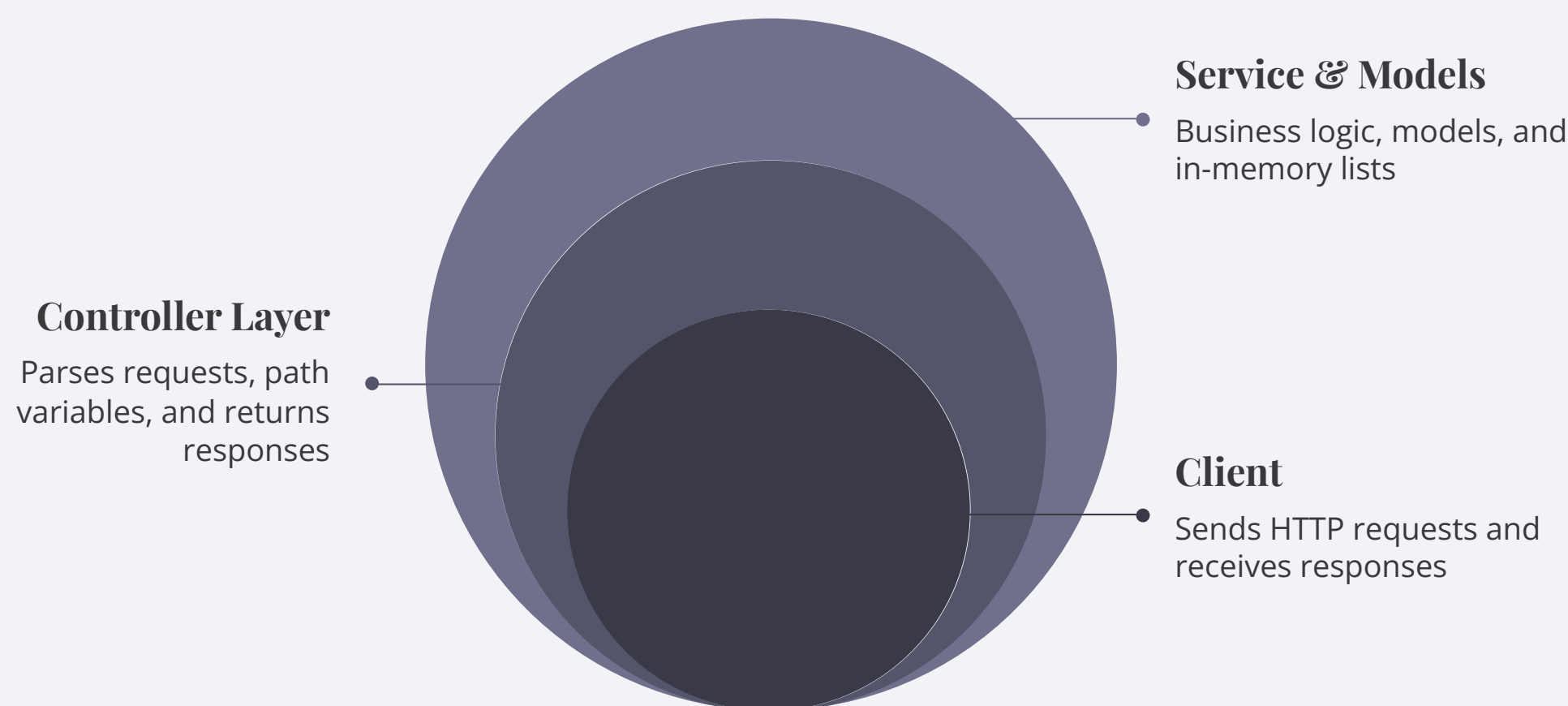
Name: Nawaf

Software Engineering Graduate

Java & Spring Boot Developer

The goal of the project was not only to implement CRUD operations, but also to build additional business logic and connect multiple services together.

System Design & Clean Code Structure



The project follows separation of responsibilities:

Controller

- Receives HTTP requests
- Reads Path Variables and Request Bodies
- Returns ResponseEntity and HTTP status codes

Service

- Contains business logic
- Searches and modifies data
- Validates relationships between entities
- Returns standardized integer result codes

Models

- Represent the system entities
- User
- Product
- Category
- Merchant
- MerchantStock

Standardized Return Approach

Used across services:

0 = Success
1, 2, 3... = Different failure cases

Then the Controller converts these cases into consistent API responses using switch expressions:

Service: `return 0 / 1 / 2 / ...`

Controller: `switch(result)`

- → 200 Success
- → 400 Bad Request

Code Reuse

Instead of repeatedly writing loops to find objects by ID, reusable methods are used:

```
getUser(id) getProduct(id) getCategory(id) getMerchant(id)
getMerchantStock(id)
```

Example:

```
Product product =
    productService.getProduct(productId);

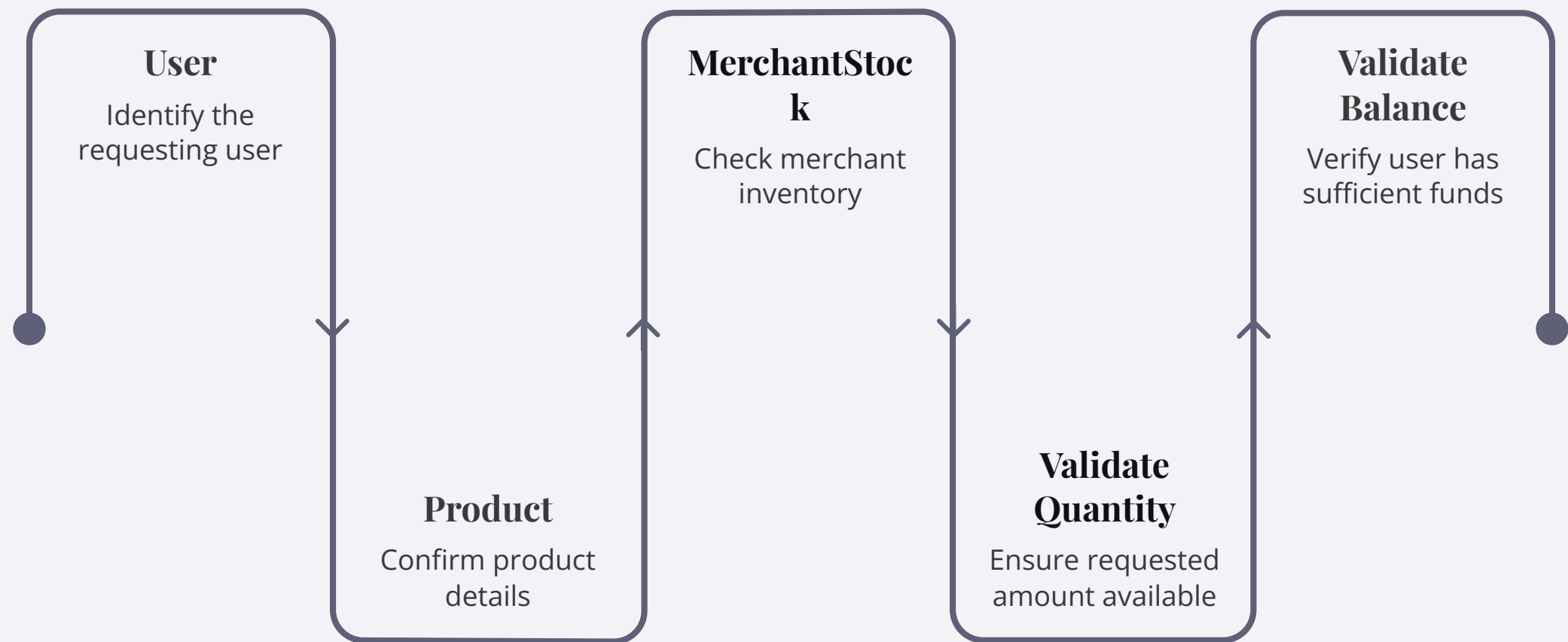
if (product == null) return 1;
```

This reduces duplicated code and makes the services easier to read and maintain.

1. Buy Product with Quantity

POST

POST /api/v1/user/buy-product/{userId}/{productId}/{merchantId}/{quantity}



Validations

- User exists
- Product exists
- Merchant stock exists
- Stock is not zero
- Requested quantity is available
- User has enough balance

Business Calculation

$totalPrice = productPrice \times quantity$

$userBalance = userBalance - totalPrice$

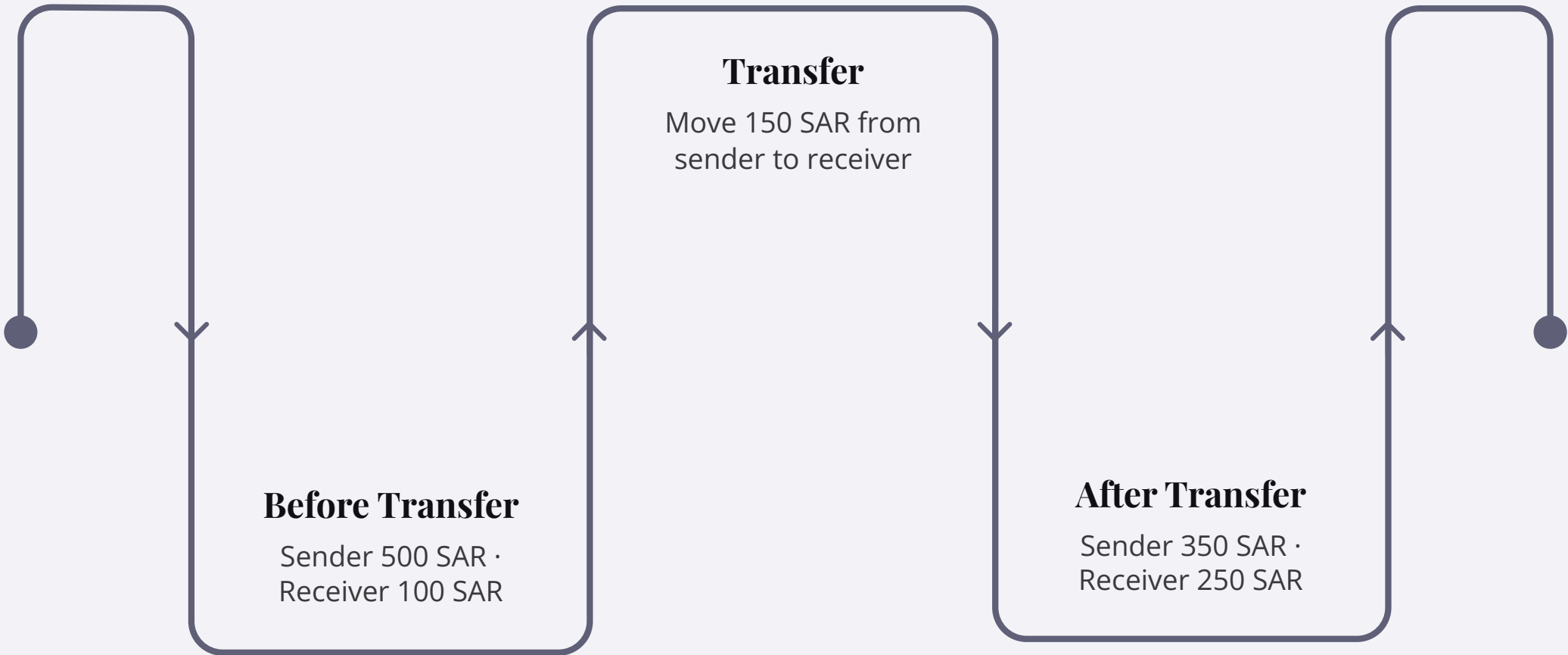
$stock = stock - quantity$

This endpoint connects **User**, **Product**, and **MerchantStock** services.

2. Transfer Balance Between Users

POST

```
POST /api/v1/user/transfer/{userId}/{receiverId}/{amount}
```



Before Transfer

Sender Balance: **500 SAR**

Transfer: **150 SAR**

Receiver Balance: **100 SAR**

After Transfer

Sender: **350 SAR**

Receiver: **250 SAR**

Validations

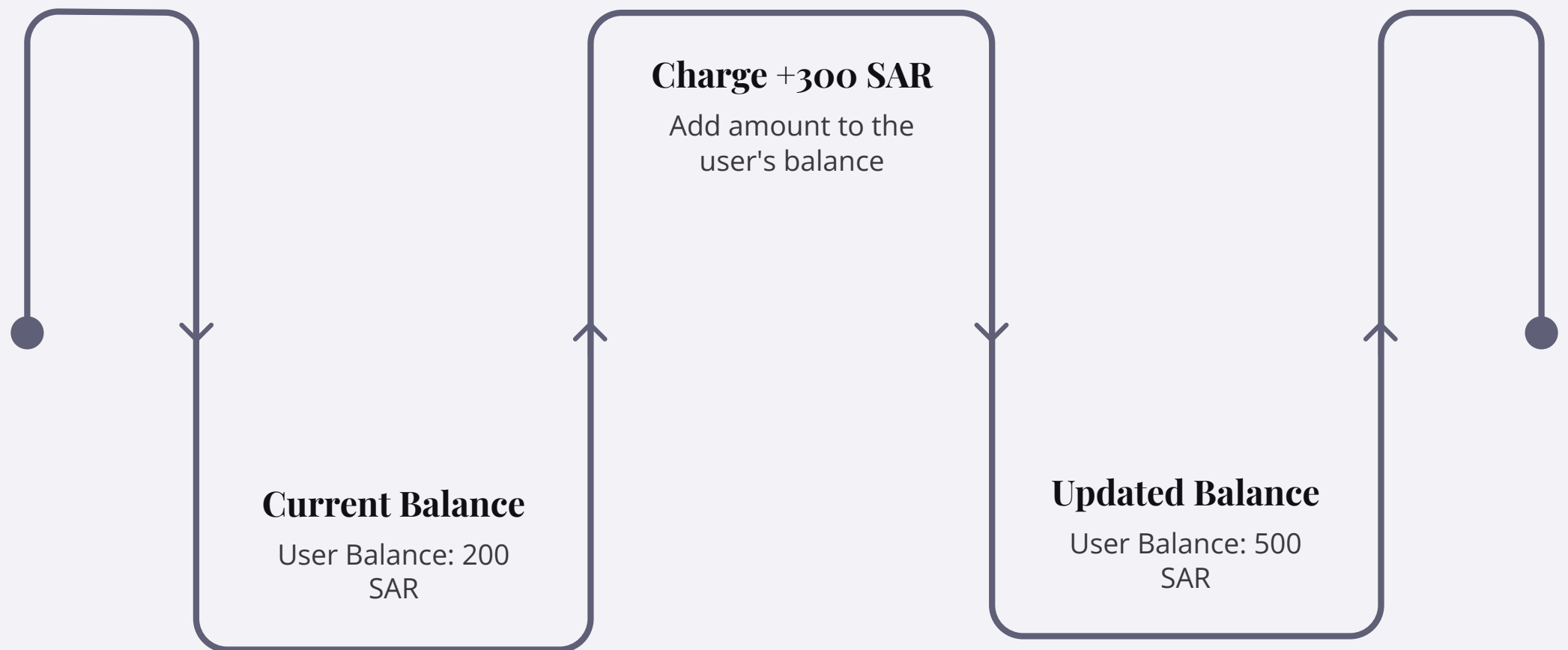
- Sender exists
- Receiver exists
- Sender and receiver cannot be the same user
- Amount must be positive
- Sender must have enough balance

Both user balances are updated in one business operation.

3. Charge User Balance

PUT

PUT /api/v1/user/charge/{id}/{amount}



Business Logic

```
newBalance = currentBalance + amount
```

Validations

- User must exist
- Amount must be a positive number

Possible Responses

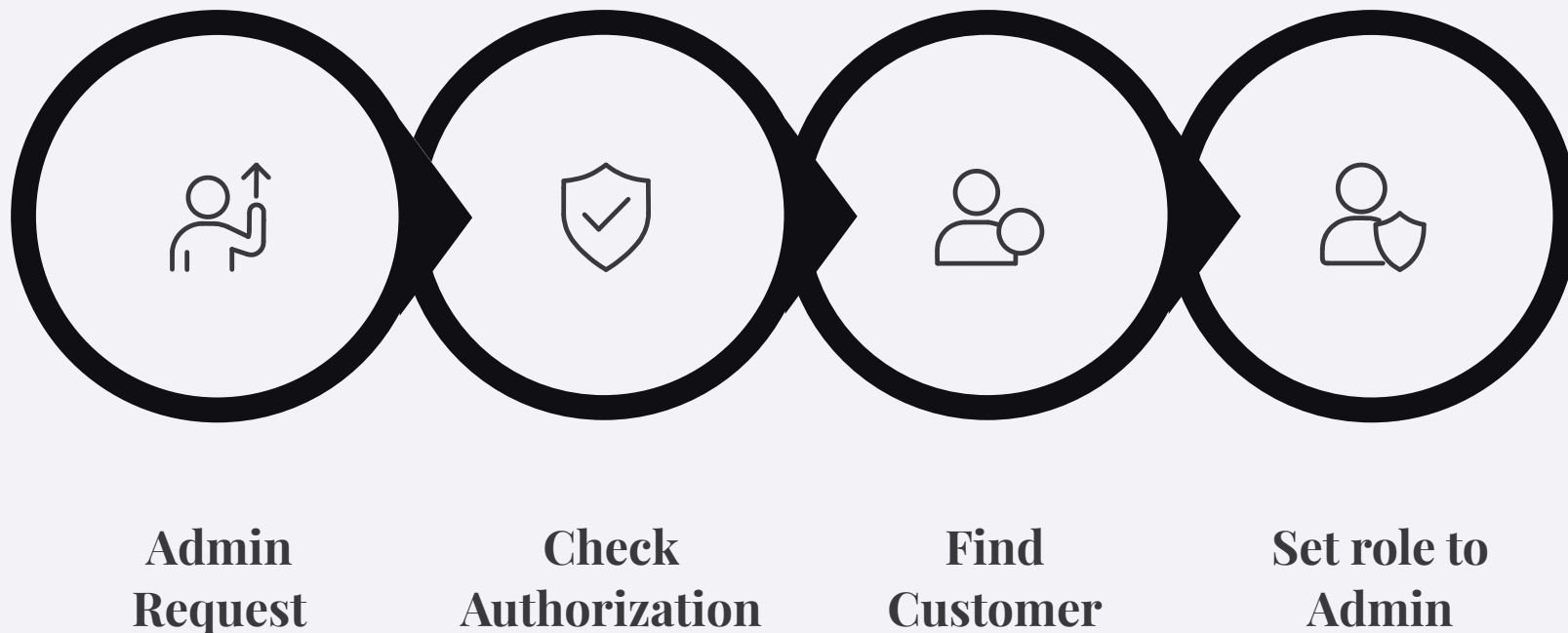
- 400 → User not found
- 400 → Amount must be positive
- 200 → Balance charged successfully

This endpoint adds money to a user's current balance using a simple charge operation.

4. Change Customer Role to Admin

PUT

PUT /api/v1/user/change-role-to-admin/{adminId}/{customerId}



Validations

- Admin exists
- Customer exists
- Requesting user must have admin role
- Customer must not already be an admin

Possible Responses

- **400** → Admin / Customer not found
- **400** → Not authorized
- **400** → User already admin
- **200** → Role changed successfully

This endpoint introduces role-based business authorization without using Spring Security yet.

5. Apply Discount to an Entire Category

PUT

PUT `/api/v1/product/discount-category/{categoryId}/{percentage}`

Example: Electronics Category

Before discount:

- Laptop: 4000 SAR
- Monitor: 1000 SAR
- Keyboard: 500 SAR

Apply **20% Discount**

After discount:

- Laptop: 3200 SAR
- Monitor: 800 SAR
- Keyboard: 400 SAR

Calculation

$$\text{percentage} = \text{percentage} / 100$$
$$\text{newPrice} = \text{price} - (\text{price} \times \text{percentage})$$

Validations

- Percentage must be positive
- Percentage cannot exceed 100
- At least one product must belong to the category

This is a **bulk business operation** because one API request can update multiple products.

Additional Business Endpoints

1	Buy Product As Gift POST POST /api/v1/user/buy-gift/{senderId}/{receiverId}/{productId}/{merchantId} Buy a product as gift to another user.
2	Discount Single Product PUT PUT /api/v1/product/discount-product/{id}/{percentage} Applies a percentage discount to one product.
3	Fix Invalid Product Categories PUT PUT /api/v1/user/products/fix-invalid-categories/{adminId} Admin maintenance operation. If a Product references a Category that no longer exists: product.categoryId = null
4	Fix Invalid Merchant Stock References PUT PUT /api/v1/user/stocks/fix-invalid-references/{adminId} Checks MerchantStock relationships. If product does not exist: productId = null. If merchant does not exist: merchantId = null
5	System Summary GET GET /api/v1/user/system-summary Returns system-level statistics such as: - Total users - Total customers - Total admins - Total products - Total categories - Total merchants - Total stock - Out-of-stock count - Average product price

Thank You

Questions?
